CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 2: Number Representations, C Intro

Instructor: Justin Yokota

Agenda

- Integer representations
  - Unsigned numbers
  - Sign-Magnitude
  - Bias
  - Two’s Complement
- C Introduction
  - Hello World!
  - Compiled vs Interpreted Languages
  - C vs Java
  - Variables and Types
  - `printf`
  - Miscellaneous Syntax

Agenda

- **Integer representations**
  - **Unsigned numbers**
  - Sign-Magnitude
  - Bias
  - Two's Complement
- C Introduction
  - Hello World!
  - Compiled vs Interpreted Languages
  - C vs Java
  - Variables and Types
  - `printf`
  - Miscellaneous Syntax

Storing Data in Binary

A system to data as binary is known as a **representation scheme**. As long we use a consistent scheme to read and write data, our program will work (even if outside users don't know our scheme).

Ideally, schemes should:

- Represent data that's relevant to whatever we're making
- Store things efficiently
  - Optimal memory use
  - Efficient operators
- Be intuitive for humans to understand
  - Can skip this one if absolutely necessary

Example: Unsigned Numbers

- The unsigned number system is the system that we've been describing up to now:
  - For n bits of data, we represent 0~(2^n-1), translating our value into its (mathematical) binary value.
  - Arithmetic operations use modular arithmetic to account for overflows
  - Called "unsigned" because we don't handle negative numbers, so we don't account for positive/negative sign

Analysis: Unsigned Numbers

- 1: Relevant data
  - Yes; you often need to store positive/nonnegative integers in your code
- 2a: Optimal Memory Usage
  - Yes; every possible bitstring yields a different, valid integer
- 2b: Efficient operators
  - Somewhat: Bitwise operations, comparators, and addition/subtraction only require O(n) transistors to construct. Multiplication, division, and modulo arithmetic require O(n^2) transistors, so they're generally slower
    - For this reason, some languages like RISC-V don't support multiplication/division operators in their base instruction set.
- 3: Intuitive
  - Yes; most aspects of this system are directly analogous to decimal math

Example: Unsigned Numbers

- The unsigned number system is the system that we've been describing up to now:
  - For n bits of data, we represent 0-(2^n-1), translating our value into its (mathematical) binary value.
  - Arithmetic operations use modular arithmetic to account for overflows
  - Called "unsigned" because we don't handle negative numbers, so we don't account for positive/negative sign
- Often used in programs when you don't need negative numbers, or when dealing with bitwise operations
- C: Built-in primitives "unsigned int", "unsigned long", etc.
- Different systems have different definitions of the size of "int"/"long"/"long long", so you can force a specific size through the alias "uint8_t", "uint16_t", etc. for 8-bit, 16-bit, etc. integers.

Problem: What if we want to handle negative numbers?

- Need to specify some encoding that includes negative values as well as positive values
- Immediate issue: we won’t be able to represent as many positive numbers
  - No free lunch: every negative number we represent is a positive number we can't represent
- For most signed systems, we want to have about as many positive numbers as negative numbers

What if we want to handle negative numbers?

How can we store signed integers?

Agenda

- **Integer representations**
  - Unsigned numbers
  - **Sign-Magnitude**
  - Bias
  - Two's Complement
- C Introduction
  - Hello World!
  - Compiled vs Interpreted Languages
  - C vs Java
  - Variables and Types
  - `printf`
  - Miscellaneous Syntax

Sign-Magnitude Numbers

- Main idea: In decimal, we write negative numbers by first writing a “-” sign, then the absolute value of that number.
  - Ex. “-1234”
  - For completeness, let’s also imagine that we use “+” for positive numbers, as in “+1234”
- Binary equivalent:
  - Use the first bit of the data to store the sign (1 is negative, 0 is positive), then use the remaining bits to store the absolute value of the number

Sign	Magnitude						
-	23						
0b1	0	0	1	0	1	1	1

Analysis: Sign-Magnitude Numbers

- 1: Relevant data
  - Yes; negative numbers get stored, with about the same number of negative values as positive
- 2a: Optimal Memory Usage
  - Somewhat; the number 0 gets two different representations (0b0000 0000 and 0b1000 0000), but otherwise, every value is unique
- 2b: Efficient Operators
  - No: comparators and addition/subtraction require additional handling for the sign bit
- 3: Intuitive
  - Yes; this is how we write numbers in decimal

Sign-Magnitude Numbers

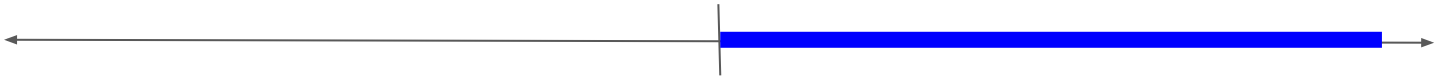
- Main idea: In decimal, we write negative numbers by first writing a “-” sign, then the absolute value of that number.
  - Ex. “-1234”
  - For completeness, let’s also imagine that we use “+” for positive numbers, as in “+1234”
- Binary equivalent:
  - Use the first bit of the data to store the sign (1 is negative, 0 is positive), then use the remaining bits to store the absolute value of the number
- Relatively uncommon as-is, but can be used as a component/starting point for more complicated systems, since as a base, it’s fairly versatile.

Agenda

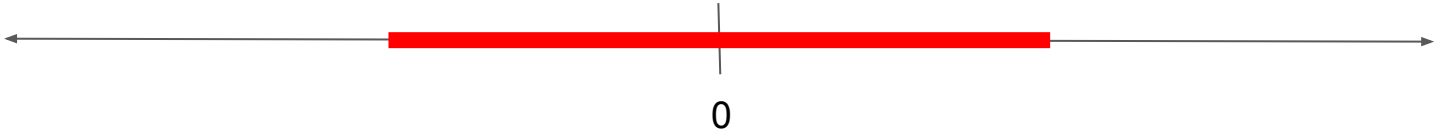
- **Integer representations**
  - Unsigned numbers
  - Sign-Magnitude
  - **Bias**
  - Two's Complement
- C Introduction
  - Hello World!
  - Compiled vs Interpreted Languages
  - C vs Java
  - Variables and Types
  - `printf`
  - Miscellaneous Syntax

Bias Numbers

- Main idea: We have a system that can represent this:

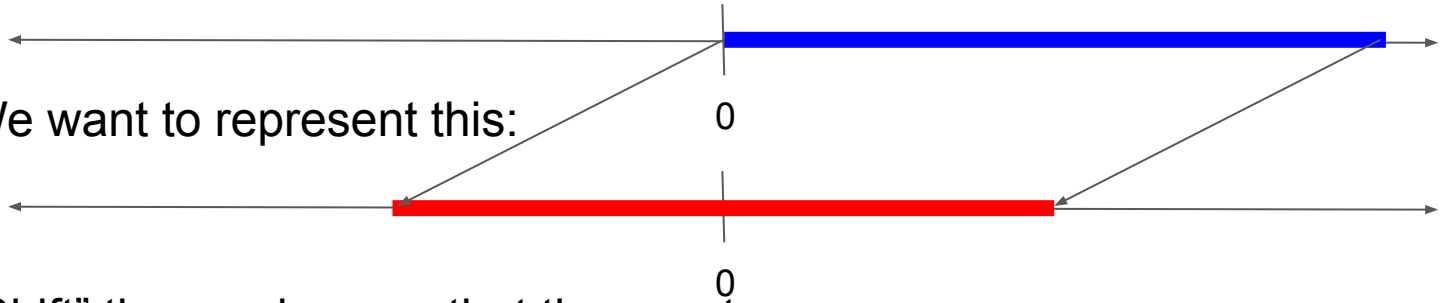


- We want to represent this:



Bias Numbers

- Main idea: We have a system that can represent this:



- We want to represent this:
- “Shift” the numbers so that they center on zero
- Formally:
  - Define a “bias”
  - To interpret stored binary: Read the data as an unsigned number, then add the bias
  - To store a data value: Subtract the bias, then store the resulting number as an unsigned number

Examples: Bias Numbers

- For the following example, let's assume that we have a bias of -127 with an 8 bit number
- If we store the binary 0b0000 1001, we mean:
  - 0b0000 1001 as an unsigned integer is 9. We then add the bias (-127) to get our value -118.
- If we want to store the number -27:
  - We subtract the bias to get a number in the range of an unsigned integer:
  - -27 - (-127) = 100
  - We then save 100 = 0b0110 0100 as an unsigned number
- Smallest number: 0b0000 0000 = -127
- Largest number: 0b1111 1111 = (255 - 127) = 128

Analysis: Bias Numbers

- 1: Relevant data
  - Yes: negative numbers get stored, with about the same number of negative values as positive
  - By choosing different values for the bias, we can represent different ranges, even those not centered at 0.
- 2a: Optimal Memory Usage
  - Yes: Every bitstring corresponds to a unique value
- 2b: Efficient Operators
  - Somewhat: Comparators stay the exact same as the unsigned numbers, since we're just adding the same number to all bit patterns
  - Any arithmetic operations become significantly harder to do
- 3: Intuitive
  - Somewhat: It makes reasonable sense, but it's harder to keep track of an extra parameter in the system

Bias Numbers

- “Shift” the numbers so that they center on zero
- Formally:
  - Define a “bias”
  - To interpret stored binary: Read the data as an unsigned number, then add the bias
  - To store a data value: Subtract the bias, then store the resulting number as an unsigned number
- Mainly used when data is only intended to be compared, not to be added/subtracted
- Really useful when the range of common values isn’t centered on zero
  - Real world example: The temperatures we see day-to-day often stay in the range of 273 K to 323 K. As such, we often think of temperatures with a bias of +273; this forms the Celsius system. Most of the time, we don’t “add” temperatures together, and just think in terms of “hotter” or “colder”. Only in chemistry/scientific fields do we often add or multiply temperatures; this is why Kelvin tends to be preferred in scientific environments.

Agenda

- **Integer representations**
  - Unsigned numbers
  - Sign-Magnitude
  - Bias
  - **Two's Complement**
- C Introduction
  - Hello World!
  - Compiled vs Interpreted Languages
  - C vs Java
  - Variables and Types
  - `printf`
  - Miscellaneous Syntax

Two’s Complement Numbers

- Main idea: When working with unsigned numbers, we noted that overflows got handled by taking the results mod 2^n.
- Why not handle negative numbers the same way?
- In binary:
  - If the leading bit of the number is 0, interpret the number as an unsigned integer (positive numbers)
  - If the leading bit of the number is 1, interpret the number as an unsigned integer, then subtract 2^n (negative numbers)
- Result: The 2’s complement value of a bitstring is always equivalent to the unsigned value (mod 2^n)

Example: Two's Complement Numbers

- 0b0001 0001
  - The leading digit is 0, so we take the unsigned value of this number, which is 17. Thus, this bitstring means 17.
- 0b1110 1111
  - The leading digit is 1, so we take the unsigned value, and subtract 2^8 = 256. The unsigned value of this number is 239, so this bitstring means 239 - 255 = -17
- There's an easy shortcut for negating numbers in two's complement: In order to multiply a number by -1, flip the bits, and add 1.
  - Ex. The number 17 is represented by the binary 0b0001 0001. If we flip the bits of this, we get 0b1110 1110. If we then add 1, we get 0b1110 1111, which is the bitstring that stores -17 in two's complement.
  - This algorithm works both ways. For example, if I start with 0b1110 1111, I can flip the bits to get 0b0001 0000, then add 1 to get 0b0001 0001 = 17, which is the negation of the -17.
- It's often useful to convert values to binary through the above sequence, but the properties of this signed number system stem from the former definition.

Analysis: Two's Complement Numbers

- 1: Relevant data
  - Yes: negative numbers get stored, with about the same number of negative values as positive
- 2a: Optimal Memory Usage
  - Yes: Every bitstring corresponds to a unique value
- 2b: Efficient Operators
  - Yes! This representation system is equivalent to unsigned numbers mod 2^n, and we've defined our operators by taking the result mod 2^n. As a result, addition, subtraction, and multiplication work exactly the same as with unsigned numbers, even using the same circuit. This makes two's complement extremely efficient to implement if you've already implemented unsigned numbers.
- 3: Intuitive
  - Somewhat: "Flip the bits and add one" is a fairly useful mnemonic that allows humans to easily translate two's complement numbers. The underlying math behind this system is fairly complex, though, so it's somewhat unintuitive why this works.

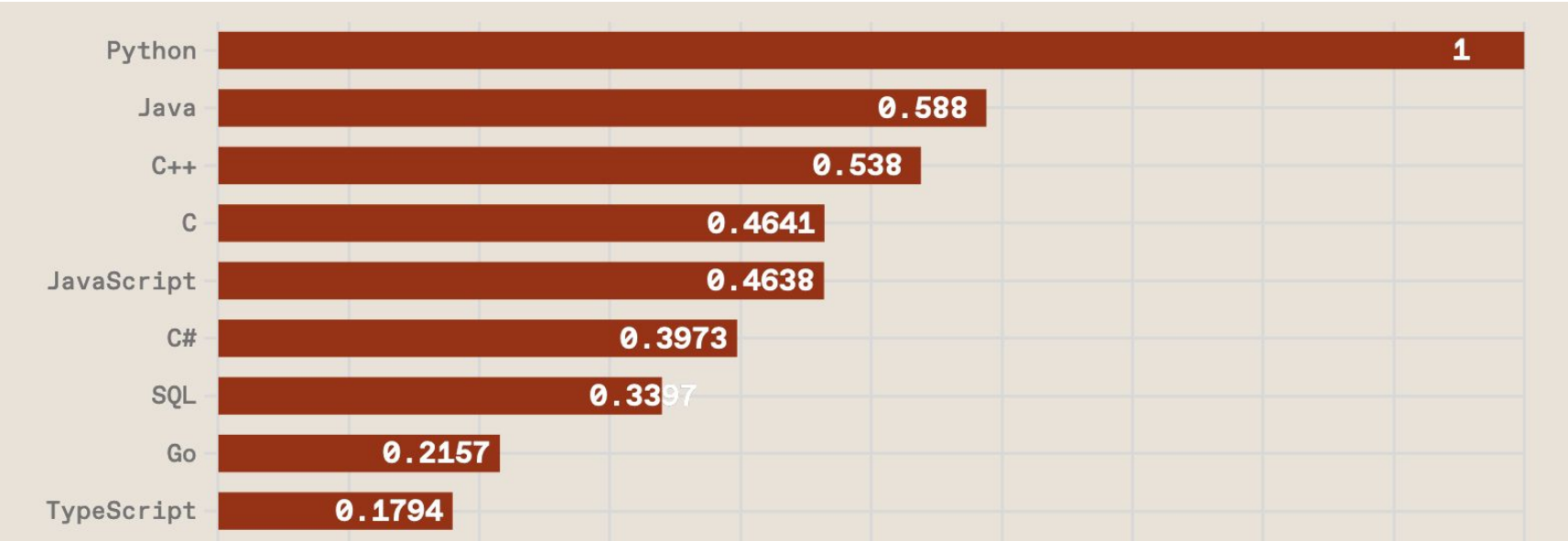
Two’s Complement Numbers

- In binary:
  - If the leading bit of the number is 0, interpret the number as an unsigned integer (positive numbers)
  - If the leading bit of the number is 1, interpret the number as an unsigned integer, then subtract 2^n (negative numbers). Alternatively, flip the bits and add one to get the magnitude of the number, and interpret the number as a negative number.
- Because it allows the entire unsigned math circuit to be reused, two’s complement is generally the implementation of choice for signed numbers. This includes C’s int, long, long long, int32_t, etc.
  - From now on, we may sometimes say “signed number” without specifying the exact format. Unless otherwise stated, this refers to a two’s complement representation.

Agenda

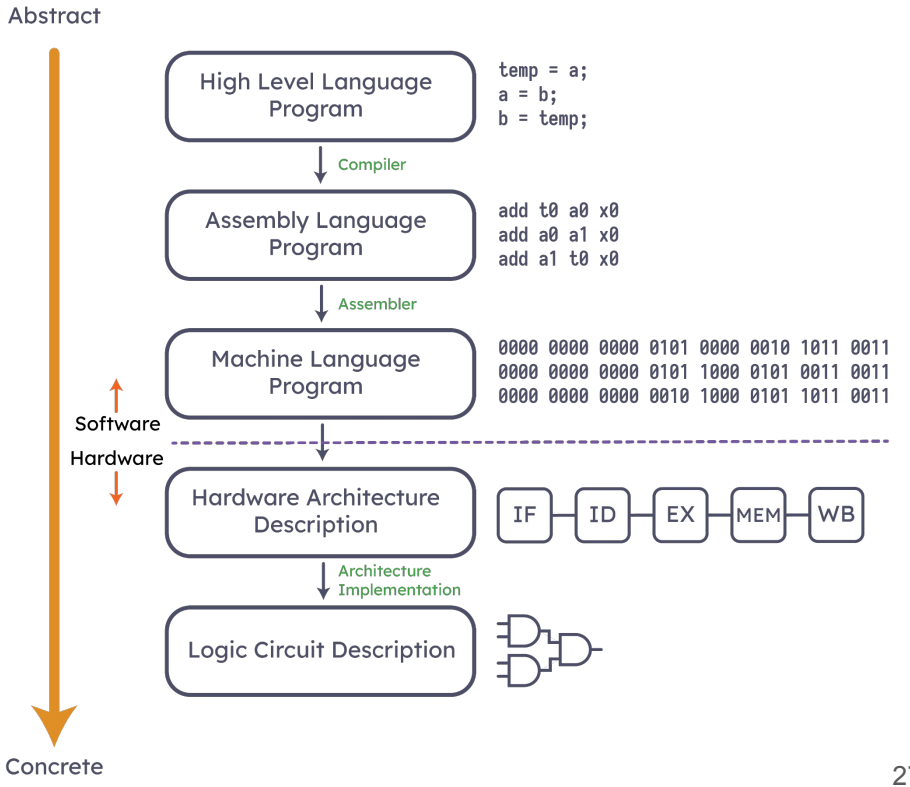
- Integer representations
  - Unsigned numbers
  - Sign-Magnitude
  - Bias
  - Two's Complement
- **C Introduction**
  - Hello World!
  - Compiled vs Interpreted Languages
  - C vs Java
  - Variables and Types
  - `printf`
  - Miscellaneous Syntax

C is useful!



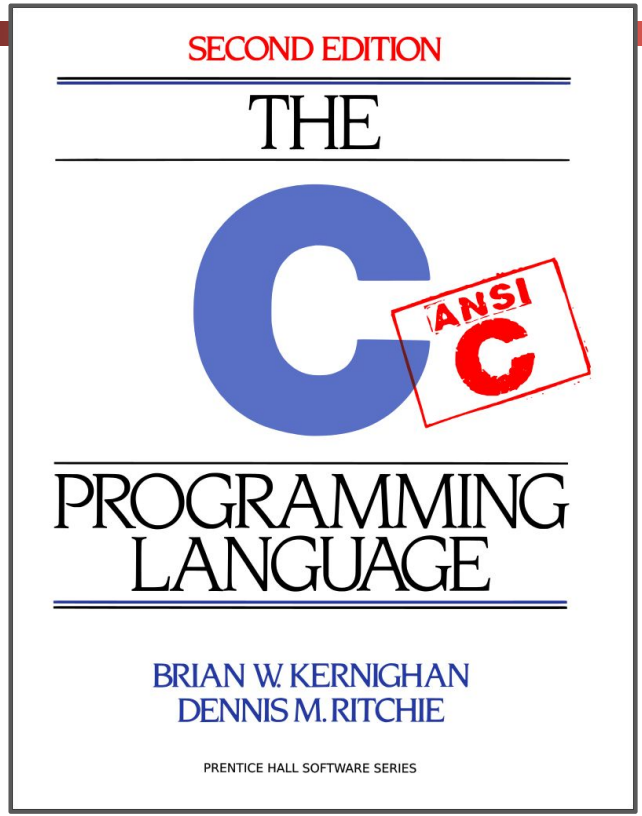
(IEEE Spectrum, 2023)

Great Idea #1 Abstraction



Why C?

- Kernighan and Ritchie (K&R)
  - *C is not a "very high-level" language, nor a "big" one, and is not specialized to any particular area of application. But its absence of restrictions and its generality make it more convenient and effective for many tasks than supposedly more powerful languages.*



Why C?

- We can exploit underlying features of the operating system
  - C gives us quite close control to the hardware
- Fun fact: the first operating system not written in Assembly was written in C!
- C has influenced several languages
  - Java syntax heavily based on C
  - C++ is just C with more features
  - C# is an "improved" version of C (very similar to Java)
- C does less for the programmer
  - It's still high-level, but it's lower than Python/Java

Disclaimers

- These lectures will not teach you how to fully code in C; rather, these lectures will teach key concepts
  - Labs, Projects, and Readings will help you apply concepts you see
  - Brian Harvey's notes from 2007: https://inst.eecs.berkeley.edu/~cs61c/resources/HarveyNotesC1-3.pdf
- C is a good language to learn lower-level programming, but isn't that practical in 2026
  - Many C concepts are inherently **unsafe**.
  - If your CS61C program contains an error, it might not crash immediately but instead leave the program in an inconsistent (and often exploitable) state.
    - Take 161/162!
- If performance matters, use Rust or Go instead
  - Rust, “C-but-safe”: By the time your C is (theoretically) correct w/all necessary checks it should be no faster than Rust.
  - Go, “Concurrency”: Practical concurrent programming takes advantage of modern multi-core microprocessors

Agenda

- Integer representations
  - Unsigned numbers
  - Sign-Magnitude
  - Bias
  - Two's Complement
- **C Introduction**
  - **Hello World!**
  - Compiled vs Interpreted Languages
  - C vs Java
  - Variables and Types
  - `printf`
  - Miscellaneous Syntax

Hello World in C


```c
#include <stdio.h>                      // Import library using #include

int main(int argc, char *argv[]) {      // Main Function
    printf("Hello World!\n");           // Print

    return 0;                           // main returns an integer
}
```

- A lot of C is in actually in libraries (e.g. `printf` from `stdio.h`)
- C is **function-oriented**
- The main function returns an integer, not void
  - 0 means success, nonzero means failure (different nonzero values can represent different failure types)
  - Main method takes in argv and argc instead of args

Running Hello World

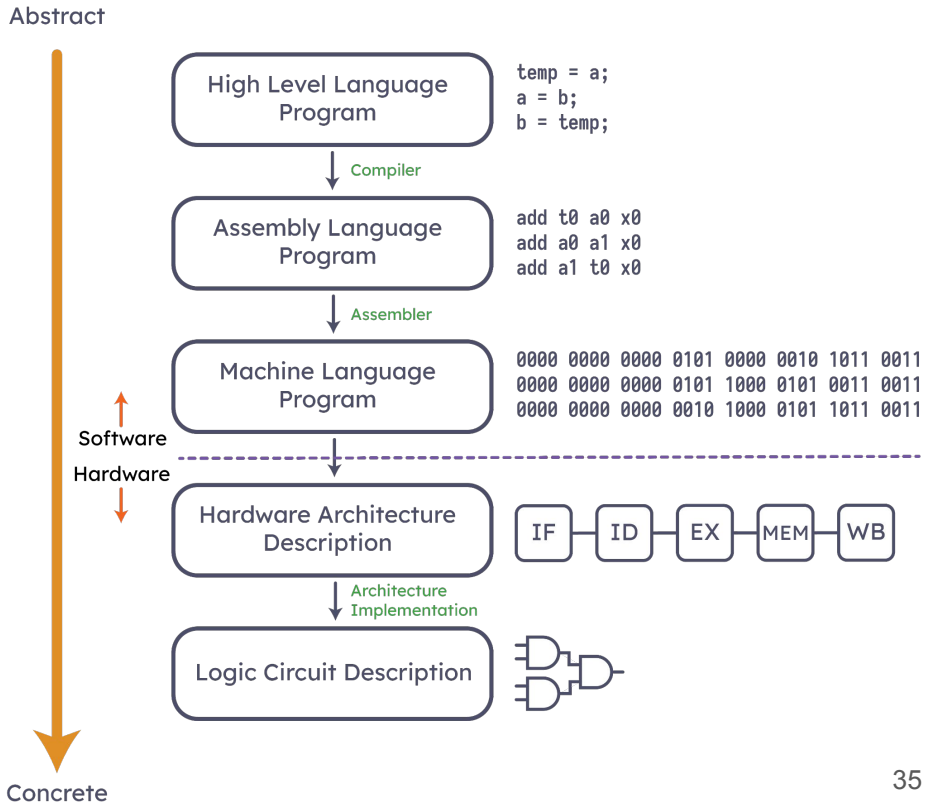

```
jusym@JY MINGW64 ~/Documents/cs61csu25
$ ls
HelloWorld.c

jusym@JY MINGW64 ~/Documents/cs61csu25
$ gcc HelloWorld.c

jusym@JY MINGW64 ~/Documents/cs61csu25
$ ls
HelloWorld.c  a.exe*

jusym@JY MINGW64 ~/Documents/cs61csu25
$ ./a.exe
Hello World
```

Agenda

- Integer representations
  - Unsigned numbers
  - Sign-Magnitude
  - Bias
  - Two's Complement
- **C Introduction**
  - Hello World!
  - **Compiled vs Interpreted Languages**
  - C vs Java
  - Variables and Types
  - `printf`
  - Miscellaneous Syntax

Compiled Languages

- C compilers
  - Map C code to **architecture-specific** machine code before the code can be run
- Compare:
  - Java converts into **architecture-independent** bytecode, which is compiled as the code runs **(JIT)**
  - Python converts into bytecode at **runtime**
- Colloquially, “compiling C” means the full process of going from source to executable
  - Actually multiple steps, with the first step being called "compiling" (more later)



Advantages

- Compiled code is quite fast, since it optimizes for a given architecture
- Compilation can usually be optimized for speed
  - Cache unchanged files
  - Parallelize compilation
  - `make` is quite good
- Note: Plenty of people do scientific computing in Python
  - Good libraries
    - numpy uses C under the hood
    - Pytorch uses C++
  - Cython allows you to call C code
  - Fun fact: Python interpreter is written in C

Disadvantages

- Compiled files need to be rebuilt on each system
  - Intel vs. Apple Silicon, etc.
  - Windows vs. Linux vs. MacOS, etc.
- This means that porting your code involves recompiling from source
- Change, Compile, Run, repeat can make development slow
  - Can be mitigated with tools such as `make` and `ninja`

Agenda

- Integer representations
  - Unsigned numbers
  - Sign-Magnitude
  - Bias
  - Two's Complement
- **C Introduction**
  - Hello World!
  - Compiled vs Interpreted Languages
  - **C vs Java**
  - Variables and Types
  - `printf`
  - Miscellaneous Syntax

C vs. Java — Philosophies

	C	Java
Type of Language	Function Oriented	Object Oriented
Programming Unit	Function	Class
Compilation	`gcc hello.c`	`javac Hello.java`
Execution	`./a.out`	`java Hello`
Storage	Manual (More on this next lecture!)	Automatic

C vs. Java — Syntax

	C	Java
Libraries	`#include <stdio.h>`	`import java.io.File`
Comments	`/* block comment */` or `// line comment`	
Variables	`int x = 5;`	
Operators	Arithmetic, Assignment: +, -, *, /, %, =, +=, -=, … Comparison: ==, !=, <=, >=, <, > Increment, Decrement: ++, -- Bitwise: <<, >>, ~, &, |, ^	
Constants	`#define` or `const`	`final`
Naming Conventions	snake_case	camelCase

C Control Flow

Pretty similar to Java:

```
int main(int argc, char *argv[]) {  // Functions have a return type, name, and arguments
    printf("Hello World!\n");
    return 0;
}

void foo() { printf(“foo\n”); }     // void works too!
```

Control also feels familiar:

```
while (expression) {
    statements;
}
```

```
for (init-statement; condition; inc-expression) {
    statements;
}
```

```
if (condition) {
    statements;
} else if
(condition) {
    statements;
} else {
    statements;
}
```

C: Use Braces

- Control flow (if-else, for, etc.) allow some omission of curly braces.
  - (as opposed to function defs., which must always have braces)
- **Single-line statements can omit** curly braces. (same as in Java, but we didn't tell you)
- ⚠️ However, subsequent lines are considered outside of the body
  - Leads to many debugging errors (StackOverflow)
  - "Just because you can, doesn't mean you should")

```
while (expression)
    statement;
```

```
for (init-statement; condition; inc-expression)
    statement; //In the for loop
    statement; //Not in the for loop
```

```
if (condition)
    statement;
else if (condition)
    statement;
else
    statement;
```

What is an argv?!?!

We've seen this as the main function:

```
int main(int argc, char *argv[]) { return 0; }
```

What are `argc` and `argv`?

- They are the command-line arguments to our program:
  - `argc` = number of strings on the command line
  - `argv` = array of strings of our arguments
- Example:

```
$ ./a.out 1 hi!
```

- `argc = 3`
- `argv = {"./a.out", "1", "hi!"}`

What is an argv?!?!


```c
#include <stdio.h>

int main(int argc, char *argv[]) {
    for (int i = 0; i < argc; i++) { // Print all arguments
        printf("%s\n", argv[i]);
    }
    return 0;
}
```

```
$ gcc arguments.c
$ ./a.out 1 hi!
./a.out
1
hi!
```

Command Line Arguments exists in Java!


```
public class HelloWorld {
   public static void main(String[] args) {
      System.out.println("hello world");
   }
}
```

args in Java works the same way as argv+argc in C

Agenda

- Integer representations
  - Unsigned numbers
  - Sign-Magnitude
  - Bias
  - Two's Complement
- **C Introduction**
  - Hello World!
  - Compiled vs Interpreted Languages
  - C vs Java
  - **Variables and Types**
  - `printf`
  - Miscellaneous Syntax

A word of caution…

C spec has *a lot* of "undefined behavior."

```
1   #include <stdio.h>                    // Import library using #include
2
3   int main(int argc, char *argv[]) {    // Main Function
4       int x;                            // Define x
5       printf("%d\n", x);                // Print x
6       return 0;                         // main returns an integer
7   }
```

What gets printed?

A: Nothing: The compiler catches the uninitialized access, and compiler errors
B: Nothing: When running the code, an error occurs
C: 0
D: A random number that changes depending on how you run the code

What gets printed?

Nothing: The compiler catches the uninitialized access, and compiler errors

Nothing: When running the code, an error occurs

0

A random number that changes depending on how you run the code

What gets printed?

0%

Nothing: When running the code, an error occurs

0%

0

0%

A random number that changes depending on how you run the code

0%

What gets printed?

0%

Nothing: When running the code, an error occurs

0%

0

0%

A random number that changes depending on how you run the code

0%

A word of caution…

C spec has *a lot* of "undefined behavior."

```
1   #include <stdio.h>                    // Import library using #include
2
3   int main(int argc, char *argv[]) {    // Main Function
4       int x;                            // Define x
5       printf("%d\n", x);                // Print x
6       return 0;                         // main returns an integer
7   }
```

```
$ gcc hello_world.c
$ ./a.out
2
```

```
$ gcc hello_world.c
$ ./a.out
0
```

A word of caution…

Undefined behaviors can…

- be unpredictable
- give different results on different computers
- give different results each time you run a program
- give the same result 99% of the time, and a different result 1% of the time
- cause “Heisenbugs”
  - random bugs that are hard to reproduce, and seem to disappear or change when debugging

We’ve seen one such case already:

- Variables **do not** have default values! We call these undefined values “garbage,” and should not be used

C Types

C is a **statically-typed** programming language.

- Variables cannot have their types changed after declaration.

```
1   int x = 4;  // Declare x as an int
2   x = 'a';    // Warning! x cannot hold a char[1]
```

- Types help the compiler and your computer determine how to read values
- Also gives more information about the data
  - How much space does this variable take up?
  - What operations does this variable support?
- Note: C will still let you assign variables to values that are the "wrong" type
  - It'll just take the binary value and treat that binary as an `int`
  - Also, chars are internally interpreted as ints anyway, so this particular case is fully fine

C Types

Type	Description	Example	Library
`int`	Signed integers	`1, -2, 0xFF`	C
`unsigned int`	Unsigned integers	`1, 9, 15`	C
`float`	Floating point decimal	`0.0, 3.14`	C
`double`	Higher precision floating point	`3.1415926`	C
`char`	Character	`'a'`	C
`long`	Longer integer	`1234567898765`	C
`long long`	Even longer integer	`1234...6061`	C
`int32_t`	32-bit signed integer	`-61`	stdint.h
`uint32_t`	32-bit unsigned integer	`61`	stdint.h

C Types

- Note: the number of bytes in an `int` depends on the computer.
- The C standard guarantees the following:

```
sizeof(int) <= sizeof(long) <= sizeof(long long)
```

- Where `sizeof` is a C operator that tells you how many bytes is in a given type.
- We recommend that you use the `intN_t` and `uintN_t` types!

It feels like we forgot a type…

What about `bool`?

- `bool` is **not** a primitive type in C!
- Instead, we have:
  - FALSE
    - `0`
    - `NULL`
  - TRUE
    - Everything else!

```
1 if (42) {
2     printf("This line will run!");
3 } else if ("Hi!") {
4     printf("This line will too!");
5 }
```

- Modern note: there's a `stdbool.h` that most people use nowadays
- C23 added `bool` as a built-in type (but we are using C17)

Agenda

- Integer representations
  - Unsigned numbers
  - Sign-Magnitude
  - Bias
  - Two's Complement
- **C Introduction**
  - Hello World!
  - Compiled vs Interpreted Languages
  - C vs Java
  - Variables and Types
  - `printf`
  - Miscellaneous Syntax

printf

- `printf` takes in as input a *format string*, which specifies the general structure of the printed line, followed by arguments.
- Example: `printf("%s\n", "Hello World!")`
- The `%s` is a *format specifier*, and will get replaced by the first argument (interpreted as a string)
- Common format specifiers

`%d`	Signed integer	`%c`	Characters
`%u`	Unsigned integer	`%f`	Floating point
`%x`	Hexadecimal, lowercase	`%%`	% sign
`%X`	Hexadecimal, Uppercase	`%e`	Scientific Notation
`%s`	Strings	`%p`	Pointers/Memory Address

```c
#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("%s\n", "Hello World!");  // %s means print a string

    int num = 1234;
    printf("Decimal:   %d\n", num);  // %d means print a decimal number
    printf("Hex:       0x%X\n", num);  // %X means print a hexadecimal number
    printf("argv[%d] = %s\n",
        argc - 1, argv[argc - 1]);   // You can use multiple arguments
    return 0;
}
```

```
$ gcc main.c
$ ./a.out
Hello World!
Decimal:   1234
Hex:       0x4D2
argv[0] = ./a.out
```

Agenda

- Integer representations
  - Unsigned numbers
  - Sign-Magnitude
  - Bias
  - Two's Complement
- **C Introduction**
  - Hello World!
  - Compiled vs Interpreted Languages
  - C vs Java
  - Variables and Types
  - `printf`
  - **Miscellaneous Syntax**

Constants, Enums, Macros

- The keyword `const` defines a constant:
  - `const int days_in_week = 7;`
  - Any variable can be declared a constant.
- `#define` is a C Preprocessor Macro
  - "Find-and-replace"
  - `#define PI (3.14159)` will *replace* every instance of `PI` with `3.14159` before compilation
- Enums: a group of options

```
1  enum cardsuit{DIAMONDS, SPADES, HEARTS, CLUBS};
2
3  enum cardsuit suit;
4  suit = DIAMONDS;
```

Adding your own Types: `typedef` and `struct`

- `typedef` allows you to define new names for existing types:
  - `typedef uint8_t BYTE;`
- `struct` allows you to define a structured group of variables

```
1  typedef enum cardsuit{DIAMONDS, SPADES, HEARTS, CLUBS} suit_t;
2  typedef struct cardstruct {
3      int rank;
4      suit_t suit;
5  } card_t;
6
7  card_t card;
8  card.rank = 1;
9  card.suit = SPADES;
```

Warnings

- Structs are similar to, but not the same as, classes
  - A Java class is a collection of variables that represent the parameters of some real-world thing, along with a set of methods that that thing can do.
  - C predates the modern concept of objects; a struct is just a collection of variables (no methods)
- Define statements can also be used to make "functions", but ultimately just does a find-and-replace.
  - Ex.
  - ```
    #define distance(x,y) x*x+y*y
    int f(int n) {
        printf("%d\n",n);
        return n;
    }
    ```
  - If you call `distance(1+2,f(4))`, the compiler replaces this with:
    `1+2*1+2+f(4)*f(4)`
    Evaluates to 21 instead of 25, and prints 4 twice
    ```